

复习

数据结构的结构定义

1. 数据的逻辑结构

二者关系

2. 数据的物理结构
(亦称存储结构)

3. 数据的运算：对存放在物理结构上的数据, 按定义的逻辑结构进行的各种操作

A. 线性结构

B. 非线性结构

A 顺序存储

B 链式存储

C 索引存储

D 散列存储

数组

线性表

栈

队列

树形结构

图形结构

线性表的定义

线性表是n个数据元素的有限序列

$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$

数据元素

a_i 的直接前趋

a_i 的直接后继

线性起点

线性终点

下标，是元素的序号，表示元素在表中的位置

$n=0$ 时称为**空表**

n为元素总个数，即表长

相互关系描述

- L的长度为 n ($n \geq 0$)，当 n 为0时，表示是空表
- 每个元素（除了第1个和最后一个元素外），有唯一的前趋和后继
- 线性表中各元素间存在着线性关系
- 均匀性: 各元素数据类型必须相同
- 有序性: 各元素是有序的，不可交换次序



插入算法步骤

- 判断插入位置是否合法
- 将第 n 至第 i 个元素后移一个存储位置
- 将 x 插入到 a_{i-1} 之后
- 表的长度+1

删除算法步骤

- 判别指定的位置是否合法
- 若合法，则将位置 $i+1$ 至 n 上的元素前移一个存储位置
- 表的长度 $- 1$

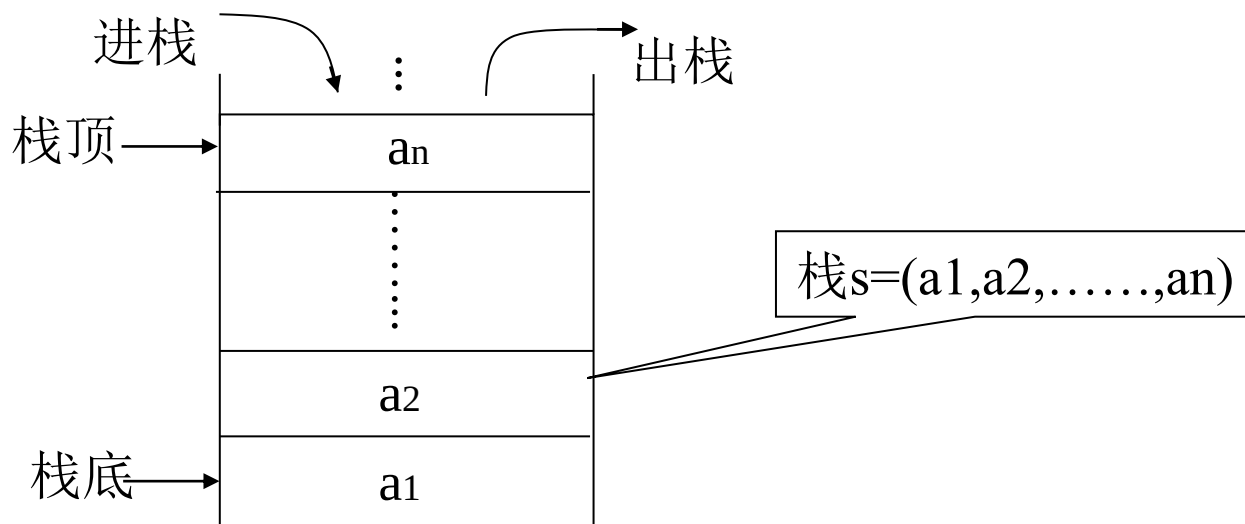
单链表

- 单链表的插入
- 单链表的删除
- 单链表的连接

栈

栈： 限定仅在**表尾**进行插入或删除操作的线性表，表尾—**栈顶**，表头—**栈底**，不含元素的空表称**空栈**

特点： 先进后出（**FILO**）或后进先出（**LIFO**）



进栈步骤

- 判别栈满否，若满，则显示栈溢出信息，停止执行；否则，执行下一步；
- 栈顶指针 top 上移(加1)；
- 在 top 所指的位置插入元素 x 。

出栈步骤

- 判别栈是否为空；若空，则输出栈下溢信息，并停止执行；
否则，执行下一步；
- 弹出（删除）栈顶元素；
- 栈顶指针top下移(减1)。

两栈共享

有时，一个程序设计中，需要使用多个同一类型的栈，这时候，可能会产生一个栈空间过小，容量发生溢出，而另一个栈空间过大，造成大量存储单元浪费的现象。

两栈共享

- 如何充分利用栈空间，这是解决存储空间紧张这样一个实际矛盾所要考虑的问题。
- 作为一种策略，就是多栈共享一个栈空间。
- **具体做法：**对两栈共享情况来说，将两个栈底分别设在两端，两个栈顶指针 $top1$ 和 $top2$ 相对中间位置动态移动，两个栈之间的分界线是不定的。

队列

队列的定义

队列 (Queue) 也是一种运算受限的线性表。它只允许在表的一端进行插入，而在另一端进行删除。允许删除的一端称为**队头 (front)**，允许插入的一端称为**队尾 (rear)**



队列的特性

- 先进入队列的成员总是先离开队列。因此队列亦称作先进先出 (First In First Out) 的线性表，简称FIFO表。

相关的概念

- 进行插入操作的端称为队尾，进行删除操作的端称为队头。
- 队列中没有元素时，称为空队列。
- 队列空的条件： $\text{front} = \text{rear}$
- 队列满的条件： $\text{rear} = \text{MAXSIZE}$

循环队列的处理思想

如果使当 $\text{rear} = \text{MAXSIZE} + 1$ 时，即超过队列末端时，令 $\text{rear} = 1$ ；从而使队列的首尾相连接，只有当队列中真正没有空位置时，才产生溢出。

用表达式描述，

```
if (rear > MAXSIZE )  
    rear = 1 ;  
else  
    rear = rear + 1 ;
```

循环队列

- 循环队列入队算法
- 循环队列出队算法

数组

数组的顺序存储结构

- 数组元素是连续存放的，因此采用顺序存储结构。
- 无论几维数组, 在计算机中都是按一维数组来存放。数组存放通常采用两种方式：
 - 按行优先顺序
 - 按列优先顺序

按行优先顺序存储结构

按行优先顺序存放是将数组看作若干个行向量。例如，二维数组 $A_{m \times n}$ ，可以看作 m 个行向量，每个行向量 n 个元素。数组中的每个元素由元素的两个下标表达式唯一的确定。

地址计算公式：

$$LOC(a_{ij}) = LOC(a_{11}) + ((i-1) * n + (j-1)) * L$$

其中， L 是每个元素所占的存储单元。

按列优先顺序存储结构

按列优先顺序存放是将数组看作若干个列向量。例如，二维数组 $A_{m \times n}$ ，可以看作 n 个列向量，每个列向量 m 个元素。数组中的每个元素由元素的两个下标表达式唯一的确定。

地址计算公式：

$$LOC(a_{ij}) = LOC(a_{11}) + ((j-1) * m + (i-1)) * L$$

其中， L 是每个元素所占的存储单元。

数组的压缩存储

实际工程问题中推导出的数组常常是高阶、含大量零元素的矩阵，或者是些有规律排列的元素。为了节省存储空间，通常是对这类矩阵进行压缩存储。

压缩的含义是：

- 相同值的多个元素占用一个存储单元；
- 零元素不分配存储单元。

特殊矩阵

- 对称矩阵
- 三角矩阵
- 带状矩阵

对称矩阵

在一个n阶方阵A中，若元素满足下述性质：

$$a_{ij}=a_{ji} \quad 0 \leq i, j \leq n-1$$

则称A为对称矩阵。

$$\text{LOC}(a_{ij}) = \begin{cases} \text{LOC}(a_{00}) + i(i+1)/2 + j & \text{当 } i \geq j \\ \text{LOC}(a_{00}) + j(j+1)/2 + i & \text{当 } i < j \end{cases}$$

三角矩阵

以主对角线划分，三角矩阵有上三角和下三角两种。

上三角矩阵：它的下三角(不包括主对角线)中的元素均为常数。下三角矩阵正好相反，它的主对角线上方均为常数。大多数情况下，常数为0。

按行优先顺序存放上三角矩阵中的元素 a_{ij} 时
 $sa[k]$ 和 a_{ij} 的对应关系是：

$$\mathbf{k} = \begin{cases} i(2n-i+1)/2 + j - i & \text{当 } i \leq j \\ n(n+1)/2 & \text{当 } i > j \end{cases}$$

按列优先？顺序存放下三角矩阵的存储和对称矩阵类似，
sa[k]和a_{i j}对应关系是：

$$k = \begin{cases} j(2n-j+1)/2 + i - j & i \geq j \\ n(n+1)/2 & i < j \end{cases}$$

带状矩阵

对角矩阵中，所有的非零元素集中在以主对角线为中心
的带状区域中，即除了主对角线和主对角线相邻
两侧的若干条对角线上的元素之外，其余元素皆为零。

以三条带为例，假设在 a_{ij} 之前有 i 行，共有
 $3*i-1$ 个非零元素，在第 i 行，有 $j-i+1$ 个非零元
素，这样，非零元素 a_{ij} 的地址为：

$$sa[k]=3*i-1+(j-i+1)= 2i+j$$

树

树的定义

定义：树(tree)是 $n(n>0)$ 个结点的有限集 T ，其中：

有且仅有一个特定的结点，称为树的根(root)

当 $n>1$ 时，其余结点可分为 $m(m>0)$ 个互不相交的有限集

$T_1, T_2, \dots, T_m$ ，其中每一个集合本身又是一棵树，称为根的子树(subtree)

特点：

树中至少有一个结点——根

树中各子树是互不相交的集合

基本术语

结点(node) 表示树中的元素，包括数据项及若干指向其子树的分支

分支度(degree) 结点拥有的子树数

叶子(leaf) 度为0的结点

孩子(child) 结点子树的根称为该结点的孩子

双亲(parents) 孩子结点的上层结点叫该结点的~

兄弟(sibling) 同一双亲的孩子

树的度 一棵树中最大的结点度数

结点的阶层(level) 从根结点算起，根为第一层，它的孩子为第二层.....

高度(depth) 树中结点的最大层次数

森林(forest) $m(m \geq 0)$ 棵互不相交的树的集合

树的表示：

- 树型表示
- 图形表示法
- 凹入表表示
- 嵌套集合表示

二叉树

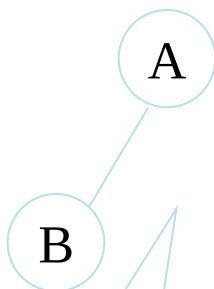
- 定义：二叉树是 $n(n \geq 0)$ 个结点的有限集，它或为空树($n=0$)，或由一个根结点和两棵分别称为左子树和右子树的互不相交的二叉树构成
- 特点
 - 每个结点至多有二棵子树(即不存在度大于2的结点)
 - 二叉树的子树有左、右之分，且其次序不能任意颠倒
- 基本形态

Φ

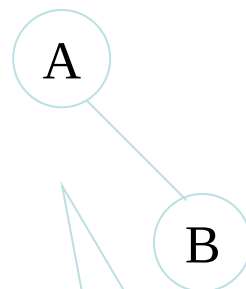
空二叉树

A

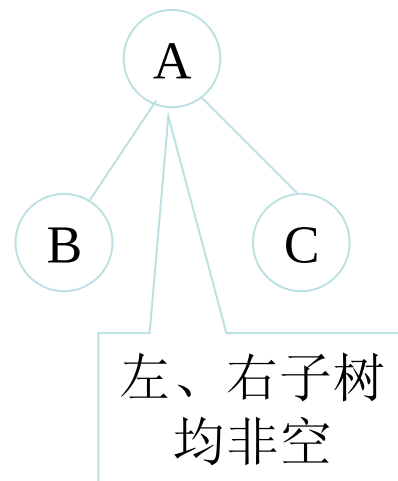
只有根结点的
二叉树



右子树为空



左子树为空

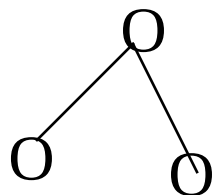


左、右子树
均非空

二叉树与树的区别（表达形式）

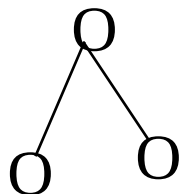
表达形式（对3个结点）

普通树

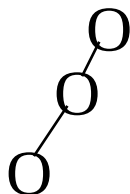


二叉树

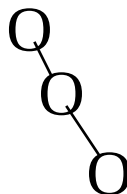
(a)



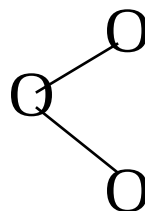
(b)



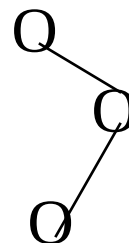
(c)



(d)



(e)



二叉树和树的区别（观念）

- 二叉树是一种特定的树结构，但不是普通树的特殊情况；
- 二叉树的结点的子树分左子树和右子树，而树则无此区分。

二叉树的五条性质

二叉树的表示方法

- 二叉树数组表示法
- 二叉树链表表示法

二叉树数组表示法

- 优点：

- 找父节点、子节点和兄弟节点容易
- 每个节点的存储空间不大

- 缺点：

当二叉树的高度和节点比例偏高时，内存利用率偏低。

二叉树链表表示法

特点：

找儿子容易，找父亲难

二叉树的遍历

- 遍历（Traversing）是树形结构的一种重要运算，即按一定的次序系统地访问结构中的所有结点，使每个结点只被访问一次。
- 遍历的方法很多，常用的有：
 - 前序法（PreOrder）
 - 中序法（InOrder）
 - 后序法（PostOrder）
 - 层次法

一棵二叉树的前序序列和中序序列（或者后序序列和中序序列）可以唯一的确定这棵二叉树。

二叉查找树

- 二叉查找树的概念
- 二叉查找树的构造
- 二叉查找树的查找
- 二叉查找树的结点删除

线索二叉树

- 线索：指向前驱或后继结点的指针称~
- 线索二叉树：加上线索的二叉链表表示的二叉树叫~
- 前驱与后继：在二叉树的先序、中序或后序遍历序列中两个相邻的结点互称为~
- 线索化：对二叉树按某种遍历次序使其变为线索二叉树的过程叫~

➤ 建立前序线索二叉树

➤ 建立中序线索二叉树

➤ 建立后序线索二叉树

哈夫曼树

- 构造哈夫曼树
- 计算哈夫曼树的带权路径长度
- 构造哈夫曼编码

树、森林与二叉树的关系

- 树、森林与二叉树的转换
- 树的遍历

排序

基本概念

- 排序分类方法：
 - 按待排序涉及的存储器不同
 - 按排序依据的原则

排序分类

- 按待排序涉及的存储器不同
 - 内排序：待排序记录存放在内存
 - 外排序：排序过程中需对外存进行访问

排序分类

- 按排序依据原则
 - 插入排序：直接插入排序、谢耳排序
 - 交换排序：冒泡排序、快速排序
 - 选择排序：简单选择排序、堆排序
 - 归并排序：2-路归并排序
 - 基数排序：

基本概念

- 排序的特性：
关键字相同的记录在排序过程中是否保持前后次序不变。

需要掌握的算法

- 冒泡排序算法
- 快速排序算法
- 选择排序算法
- 插入排序算法

堆排序

- 如何建堆？
- 如何进行堆排序？

查找方法

静态查找：线性查找、折半查找、分块查找

动态查找：树型查找、杂凑查找

查找方法评价

• 查找方法评价：

➤ 查找速度

➤ 占用存储空间多少

➤ 算法本身复杂程度

➤ **平均查找长度ASL**(Average Search Length):
为确定记录在表中的位置，需和给定值进行比较的关键字的个数的期望值叫查找算法的**ASL**。

对含有 n 个记录的表， $ASL = \sum_{i=1}^n p_i c_i$

其中： p_i 为查找表中第 i 个元素的概率， $\sum_{i=1}^n p_i = 1$

c_i 为找到表中第 i 个元素所需比较次数

分块查找

- 算法思想：

首先，查找索引表，因为索引表是有序表，故可采用二分查找，也可以采用线性查找，以确定待查的结点在哪一块；然后在以确定的那一块中顺序查找。

- 适用条件： 分块有序表

分块查找

- 掌握分块查找的算法
- 求分块查找的ASL

动态查找---杂凑查找

- 基本思想和目标
- 碰撞和同义词的概念
- 杂凑函数的构造方法
- 处理冲突的方法

杂凑查找

- 基本思想：在记录的存储地址和它的关键字之间建立一个确定的对应关系；这样，不经过比较，一次存取就能得到所查元素的查找方法。
- 目标：提高查找效率，即缩短查表和填表的时间。

处理碰撞的方法

- 线性开放寻址法
 - 线性探测法
 - 二次方探索法
 - 双哈希函数法
- 差值解决法
- 链表解决法
- 分桶杂凑法

图

图的分类

- 有向图 (Digraph)

图G中顶点的偶对若是有向的，形成的图称为有向图。
为示区别，其偶对用 $\langle v_x, v_y \rangle$ 表示。

- 无向图 (Undigraph)

图G中顶点的偶对若是无向的，形成的图称为无向图，
其偶对用 (v_x, v_y) 表示。

图的表示法

在图的表示中，常用下面两种方法：

- 邻接矩阵表示法
- 邻接表表示法

图的搜索

- 图的搜索 (Traversing Graph)

从图中指定顶点出发访问图中每一个顶点，且使每个顶点只被访问一次，该过程为图的搜索。

- 图的搜索要比树结构复杂的多。出发点不同，任一顶点的邻接点就不相同，路径也就不同。
- 常用的图的搜索方法有两种：
 - 深度优先法
 - 广度优先法

图的应用

- 最小生成树
- 拓扑排序

最小生成树

- 按照生成树的定义， n 个顶点的连通网的生成树有 n 个顶点、 $n-1$ 条边。
- 构造最小生成树的准则：必须只使用该网络中的边来构造最小生成树；必须使用且仅使用 $n-1$ 条边来联结网络中的 n 个顶点；不能使用产生回路的边。

最小生成树

- 该问题是构造连通图的最小代价生成树问题。一棵生成树的代价就是树上各边（弧）的代价之和。
- 例如，若要在 n 个城市间建立通信联络网，则只需 $n-1$ 条线路。但在 n 个城市间，最多可能架设 $n(n-1)/2$ 条线路，选择哪 $n-1$ 条线路，使费用最少。
- 实现方法：
 - 普里姆（Prim）算法
 - 鲁斯卡尔（Kruskal）算法

拓扑排序方法

- 方法步骤

- 在有向图中选一个没有前驱的顶点并输出；
 - 从图中删除该顶点和所有以它为尾的弧；
 - 重复前2步, 直到全部顶点均输出为止。
 - 输出序列即为拓扑排序序列。
- 拓扑排序序列不唯一。

题型

- 选择题（20分）
- 填空题（10分）
- 简答题（40分）
- 程序填空题（10分）
- 算法题（20分）